

Sistemas Operativos

1er Recu 1º Parcial 2C2025 – Resolución

Aclaración: La mayoría de las preguntas o ejercicios no tienen una única solución. Por lo tanto, si una solución particular no es similar a la expuesta aquí, no significa necesariamente que la misma sea incorrecta. Ante cualquier duda, consultar con el/la docente del curso.

Teoría – Single Choice

- 1- a) Al finalizar de ejecutar la instrucción en curso
- 2- d) División por cero
- 3- d) Ninguna es correcta
- 4- d) Perjudica a los procesos CPU bound
- 5- d) Todas son correctas
- 6- a) No desalojo
- 7- c) Requiere que los procesos indiquen sus peticiones máximas
- 8- a) El livelock es generalmente más difícil de detectar

Teoría – Desarrollar

1. Hay varias alternativas. Una podría ser:

```
int global = 1; // variable global -> datos
int main() { // nueva entrada del stack
    int local = 2; // variable local "local" en stack
    char* nombre = (char*) malloc(sizeof(char) * 10);
    // variable local "nombre" en stack apunta a espacio de heap
}
```

2.
 - I/O Bound: Virtual Round Robin los favorece, ya que cuando los procesos no utilizan todo su quantum y vuelven de E/S les otorga el quantum remanente y mayor prioridad. Round Robin y FIFO tratan a todos por igual, aunque FIFO podría perjudicarlos al no tener límite de ejecución para procesos CPU bound.
 - Inanición: Ninguno tiene. RR está basado en FIFO por lo que no sufre de inanición. VRR a pesar de tener una cola de mayor prioridad, a los procesos que se encuentran en la misma les da un quantum que tiende a cero, es decir, les da la parte de su quantum anterior que no consumieron, por lo que en algún momento se les acabará su remanente y serán desalojados a la cola RR de menor prioridad.
 - T.Rta promedio: Virtual Round Robin favorece a los IO Bound, con lo que el tiempo de respuesta mejora respecto a Round Robin. En FIFO podría verse muy perjudicado dado que un proceso que acaba de volver de E/S puede esperar mucho tiempo si se está ejecutando un proceso CPU bound.

3.

- Finalizar todos los procesos asociados al deadlock -> Su implementación es sencilla pero tiene un mayor costo de negocio ya que potencialmente estaría finalizando procesos de más.
- Finalizar de a un proceso hasta que se solucione el deadlock -> Tiene un menor costo pero requiere mayor trabajo (elegir a la víctima, volver a correr el algoritmo de detección y repetir hasta que se solucione el deadlock)
- Desalojar recursos -> Es el más eficiente en términos de no finalizar procesos, pero requiere la posibilidad de devolver los procesos expropiados a un estado en que puedan reanudar su ejecución, por otro lado, puede causar inanición si un proceso es continuamente elegido como víctima.

Práctica

1.

	Est Anterior	R Anterior	Estimacion	E/S	Estimacion
P1	2	3	2.5	1	2.25
P2	2	1	1.5	1	3.75
P3	5	2	3.5	3	2.25
P4	8	1	4.5	1	3.75

P1									IO			X												
P2							IO									X								
P3										IO	IO	IO		X										
P4																			IO					X
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	

2. El SO está utilizando VRR con un $Q=3$, esto puede evidenciarse en $t=9$ y $t=10$ donde se desaloja a P1 y P2 al haber ejecutado 3 unidades de tiempo entre las 2 ráfagas, y luego sigue ejecutando P4 por fuera del orden de llegada ya que P3 utilizó todo su quantum previamente.

3.

```
terminalLibre = 1
```

```
inputEscrito = 0
```

```
ticketListo = 0
```

```
mutexTurnos[3] = {1, 1, 1}
```

```
hayTurnos[3] = {0, 0, 0}
```

llamado[100] = {0, 0,}

Corredor (100 instancias)	Terminal de Tickets (1 instancia)	Empleado (3 instancias)
<pre>acercarseATerminal() Wait(terminalLibre) escribirIdCarreraEnInput() Signal(inputEscrito) Wait(ticketListo) ticket_t ticket = recibirTicket() Signal(terminalLibre) Wait(llamado[ticket.número]) posicionarseCarrera()</pre>	<pre>while(true) { Wait(inputEscrito) id = leerInput() ticket_t ticket = generarTicket(id) entregarTicket() Signal(ticketListo) Wait(mutexTurnos[id]) agregar(ticket.número, turnos[id]) Signal(mutexTurnos[id]) Signal(hayTurnos[id]) }</pre>	<pre>while(true) { Wait(hayTurnos[getId()]) Wait(mutexTurnos[getId()]) numLlamar = retirar(turnos[getId()]) Signal(mutexTurnos[getId()]) llamarCorredor() Signal(llamado[numLlamar]) }</pre>

4.

En Usuario:

- El wait de "mutexPosts" debería estar luego de "generarPost", para que cada Usuario no se bloquee innecesariamente incluso antes de generar el post.
- El signal de "mutexPosts" debería estar inmediatamente luego de "postear" para que la sección crítica sea lo más chica posible.

En Analizador:

- El orden de los wait debería ser al revés ya que los mismos podrían generar un deadlock.
- Luego de "obtenerPost" se debería realizar el signal inmediatamente para no bloquear innecesariamente al resto de los procesos mientras se realiza una operación larga en disco.

5.

- a) Considerando dos procesos de los N (P1 y P2), como la función `get_recursos()` devuelve un recurso de forma aleatoria, puede que se presente la siguiente situación:

Proceso #1	Proceso #2
<code>while(1){</code>	<code>while(1){</code>
<code>get_recursos(); // devuelve y asigna R1</code>	<code>get_recursos(); //devuelve y asigna R2</code>
<code>get_recursos(); // se bloquea por R2 ocupado</code>	<code>get_recursos(); // se bloquea por R1 ocupado</code>
<code>//...</code>	<code>//...</code>
<code>}</code>	<code>}</code>

Es decir, puede que el proceso #2 pida un recurso ya asignado al proceso #1, provocando que se quede bloqueado esperando al mismo; y que el proceso #1 pida un recurso asignado al proceso #2, provocando que se bloquee a la espera del mismo.

- b) Una forma de solucionar el deadlock para su prevención, es modificando la función `get_recursos()` para que este devuelva los recursos de forma ordenada y no aleatoria, evitando que se produzca la espera circular (condición necesaria para la existencia de deadlock)